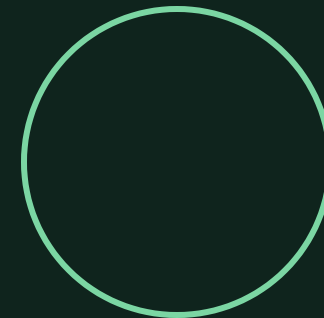


Trustworthy AI Software Modernization



From risky rewrites to tested, measured migration

Demo: Python to Rust with PyO3

Evidence: screenshots + benchmarks

Vision: language-agnostic modernization

AI should help valuable software survive technological change.

The Problem



Important software often survives longer than the technology stack it was built on.

- Rewrites are expensive, slow, and risky
- Behavior is hidden in edge cases, not documented
- Small organizations often lack the time or specialists to modernize systems
- AI can write code, but unvalidated AI rewrites are hard to trust

The modernization gap: AI can accelerate migration, but only transparent, tested, and measurable workflows can make it safe and accessible for everyone.

Why Modernize Code?

Migration is not about chasing a newer language. It is about making useful software cheaper, safer, and easier to keep alive.

Price

Moving from expensive proprietary ecosystems, can lower licensing barriers.

Sustainability

Moving code to more efficient languages can reduce CPU time, lowering energy use and cloud cost for repeated workloads.

Longevity

Updating older code to newer languages, runtimes, and package formats extends its useful lifetime.

Security

Updating the language can reduce runtime errors in performance-critical code.

Our Approach

We turn code migration into an engineering workflow instead of a one-shot prompt.

Preserve behavior first

Generate pytest suites against the original project before replacing implementation.

Reimplement with guardrails

Translate into a Rust PyO3 extension while preserving the Python-facing API.

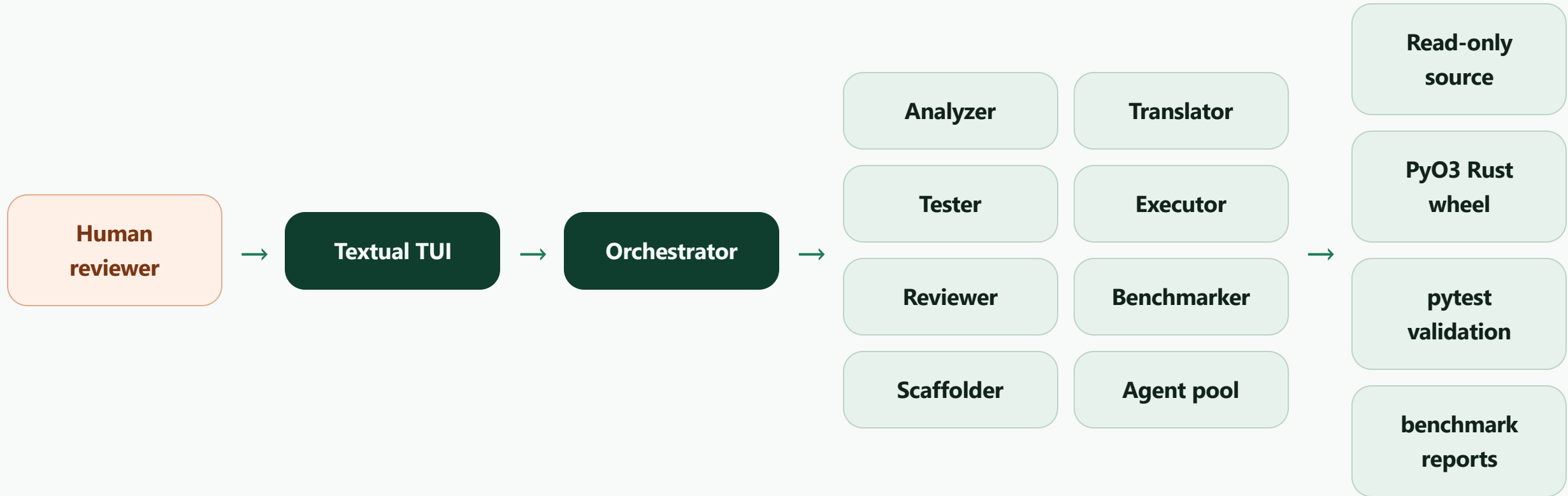
Validate with real tools

Run linting, Rust quality gates, maturin builds, installed-wheel pytest, and fix loops.

Measure the result

Benchmark isolated Python and Rust wheel installs and produce graph/report artifacts.

Architecture



Safety rule: the original source project is read-only. Generated tests, code, and measurements go to separate output folders.

Migration Workflow

1. CAPTURE BEHAVIOR

Analyze the Python project and generate pytest that documents current behavior.

2. HUMAN REVIEW

Reviewer summarizes coverage, risks, and suggested focus before approval.

3. SCAFFOLD RUST

Create a PyO3/maturin project that preserves the Python import surface.

4. TRANSLATE BEHAVIOR

Implement Rust behind the same Python-facing API.

5. VALIDATE WHEEL

Build with maturin, install the wheel, and run the same pytest suite.

6. MEASURE IMPACT

Compare isolated Python and Rust wheels and produce reports and graphs.

Why Python to Rust?



Python is accessible and widely used. Rust is fast, memory-safe, and efficient.

That makes Python-to-Rust a strong first case study:

- Keep the Python API that users already know
- Move performance-critical internals to Rust
- Build a distributable wheel with PyO3 and maturin
- Validate that behavior still matches the original package
- Measure whether the migration actually improves performance

The point is not only Rust. The point is trustworthy modernization and measurable improvements.

Beyond Python to Rust



Python to Rust is the first proof point, not the limit.

What stays the same

Analyze, test, review,
translate, validate, repair,
measure.

What changes

Language prompts,
project scaffold, package
tools, test runner, and
benchmark adapter.

What this enables

Language-agnostic
migration profiles for
different modernization
goals.

Examples: MATLAB to Python for cost, Python to Rust for efficiency, JavaScript to TypeScript for maintainability.

Current State

Implemented in the repo today:

Workflow

Six-step pipeline: tests, review, PyO3 translation, review, wheel validation, benchmarking.

Agents

Specialized roles for analysis, testing, review, scaffolding, translation, execution, and measurement.

Quality gates

flake8, mypy, baseline pytest, cargo fmt, cargo clippy, maturin build, installed-wheel pytest.

Proof artifacts

Screenshots, benchmark reports, CSV files, metadata, and generated graphs.

Demo Evidence



The system runs end-to-end inside the TUI

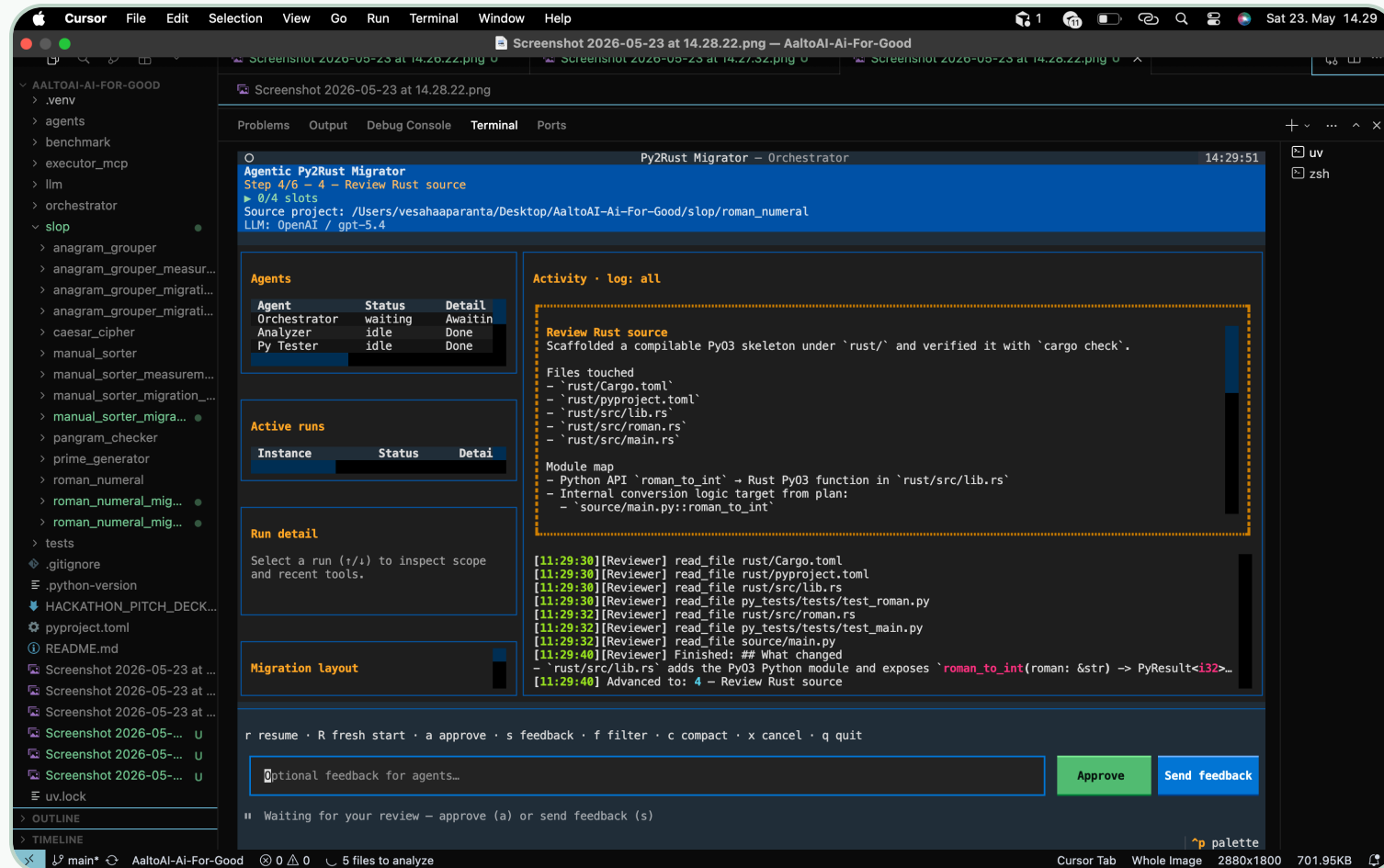
Human review

Agent logs

Benchmark completion

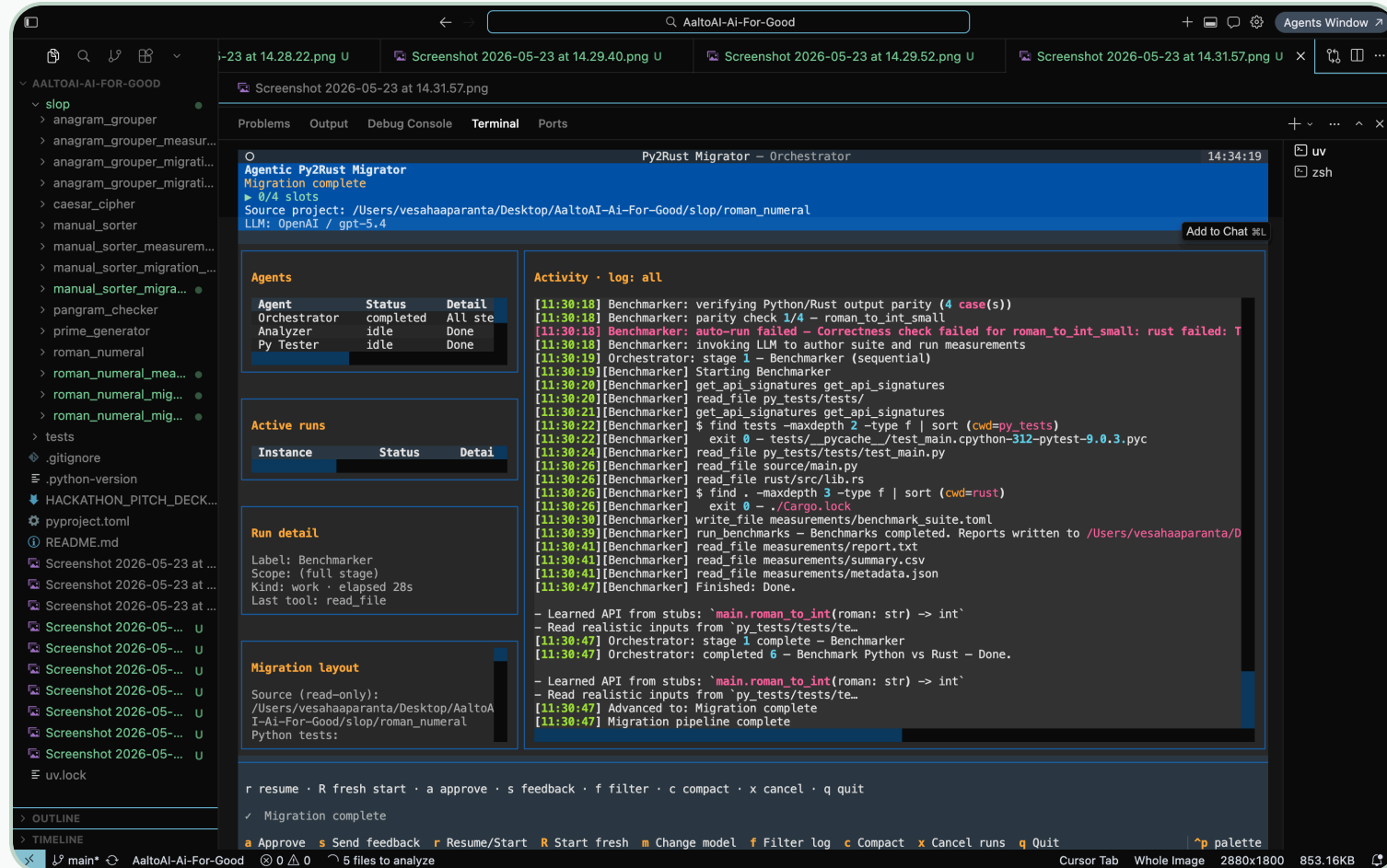
The next slides show actual demo artifacts from this repo.

Demo: Human Review Gate



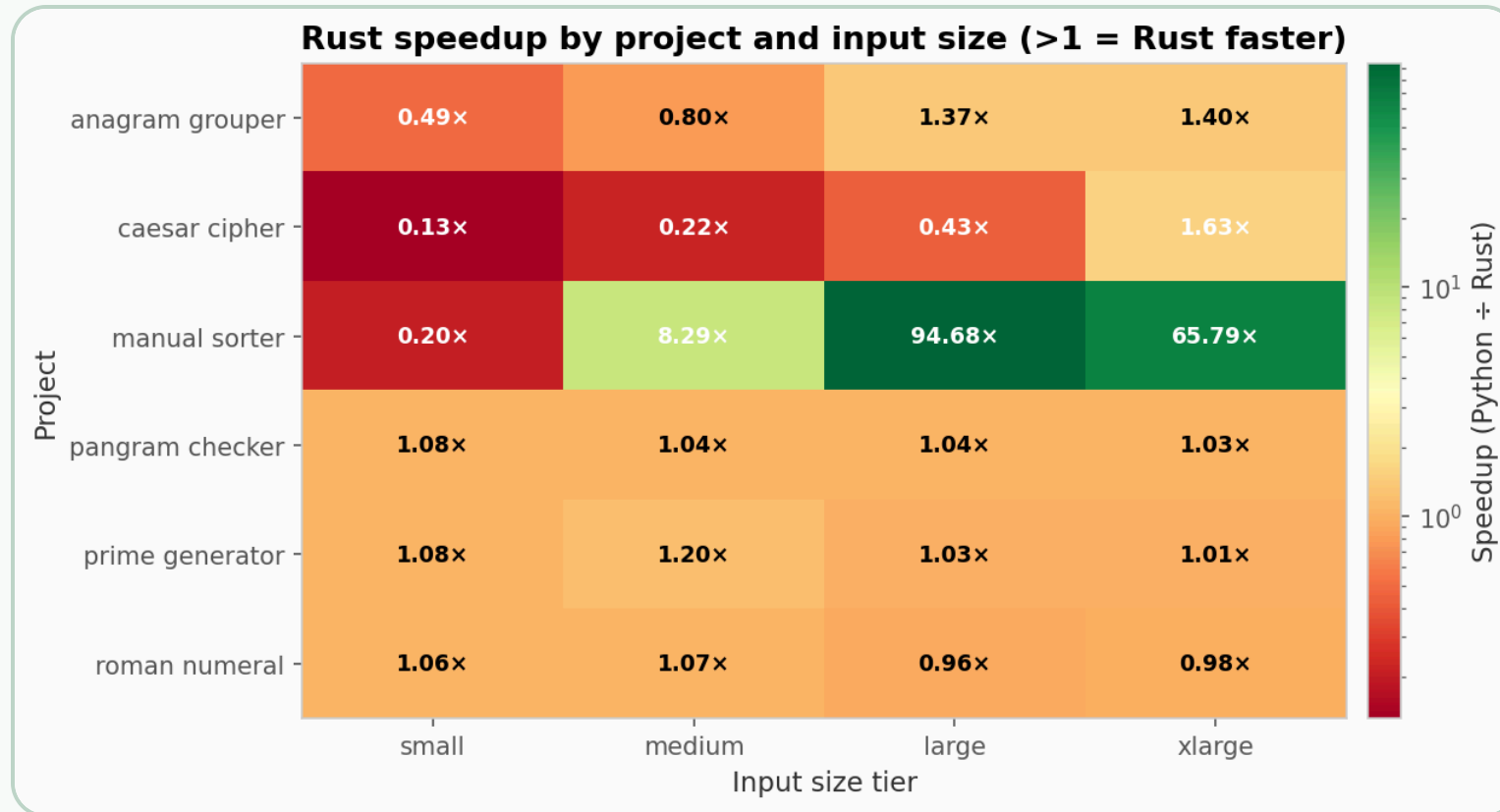
The reviewer summarizes generated Rust/PyO3 source before the user approves or sends feedback.

Demo: Migration Complete



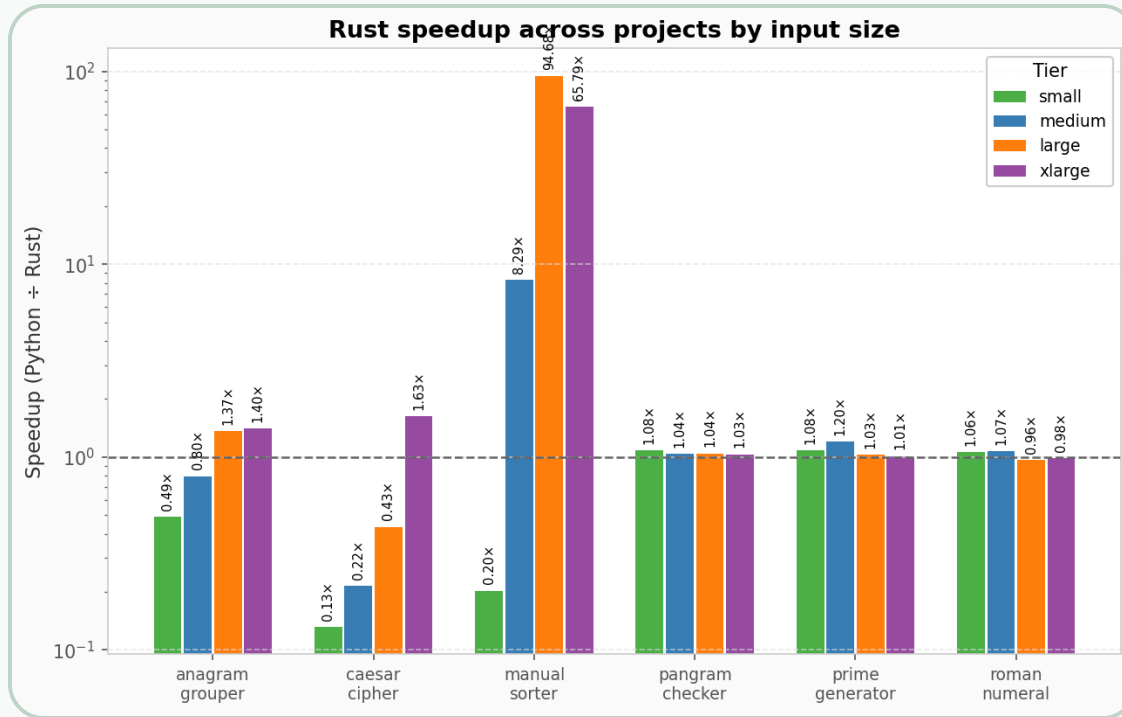
After validation, the benchmarker runs and the migration pipeline completes with report artifacts written to disk.

Demo: Measured Speedup

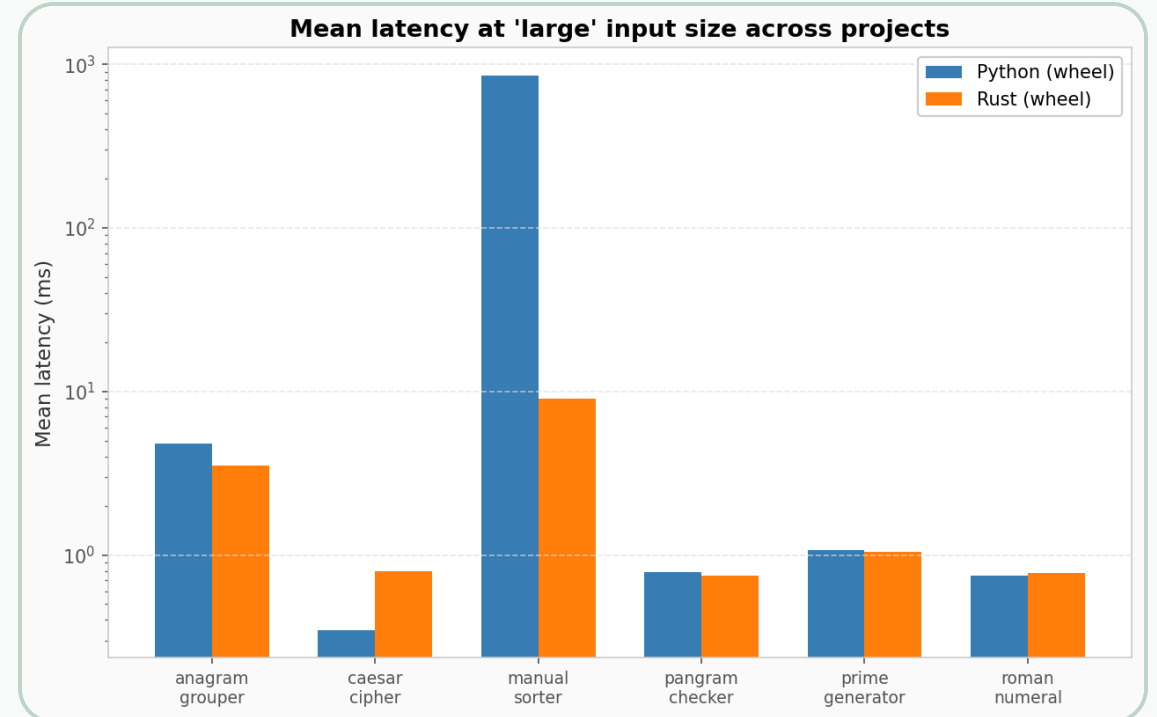


Combined benchmark heatmap across six demo projects. Green means Rust is faster; red means Python is faster.

Demo: Speedup Varies By Workload

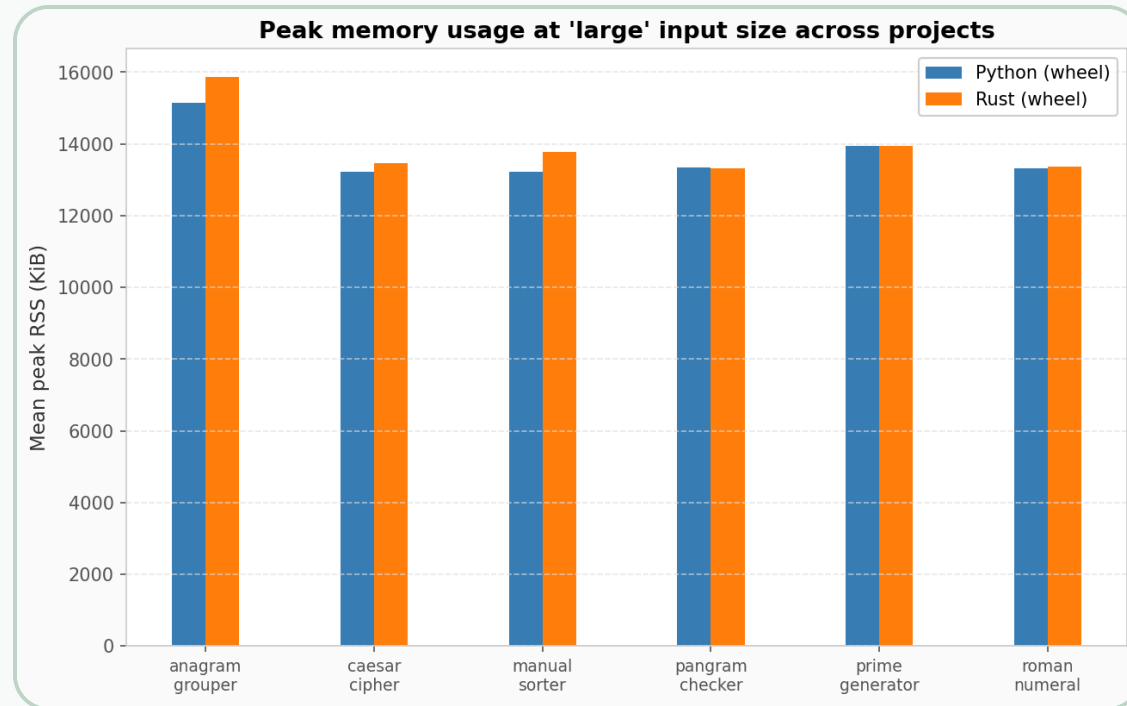


Manual sorter shows the biggest win, reaching 94.68x on large inputs and 65.79x on xlarge inputs.

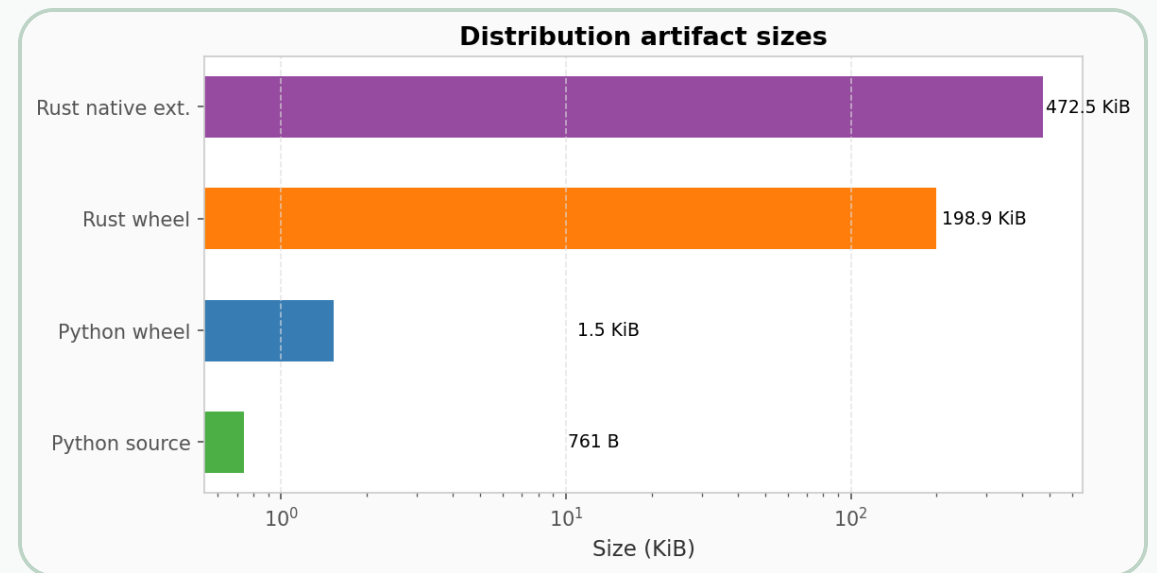


Large-tier latency shows where migration clearly helps and where the result is closer to parity.

Demo: Resource Evidence



Peak memory comparison keeps the pitch honest: performance wins may come with resource tradeoffs.



Single-project artifact sizes show package tradeoffs alongside runtime improvements.

Presentation Summary



The pitch in one sentence:

Modernization should be accessible, trustworthy, and measurable.

Why it matters

Lower cost, less compute, longer software lifetime, safer systems.

What we built

Agentic migration with tests, review gates, validation, and benchmarks.

What the demo proves

Python libraries can be translated and made into a wheel that are measurably more efficient and we proved we can translate real world examples.

Where it goes next

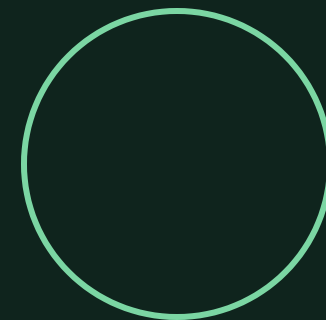
The same workflow can become language-agnostic modernization and apply to even more complex environments

Ask / Next Steps

Important questions for next steps:

- Which language pairs create the most social impact?
- What evidence makes users trust an AI-assisted migration?
- Which benchmark metrics matter most: latency, cost, energy, memory, or package size?
- How should human review work for non-expert maintainers?

Hackathon goal: prove trustworthy Python-to-Rust modernization, then generalize the pattern into migration profiles for many language pairs.



Closing

AI should help software survive technological change.

Not just generate new code, but preserve behavior, reduce migration risk, and extend the life of important systems.

Agentic Code Migration for Good

Test-driven. Human-reviewed. Benchmarked. Built for trustworthy modernization.